Facultad de Ingeniería
Departamento de Ingeniería en Computación e Informática

Analisis e interpretacion: Algoritmos iterativos de busqueda y ordenamiento

Franco Aguilar
Ivan Mansilla
Ayrton Morrison
Ingeniería Civil en Computación e Informática

Docente: Grado. Jackeline Aldridge
Ramo: Diseño de Algoritmos

2026

RESUMEN

Este informe presenta el desarrollo de un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV. Para ello, se implementaron los algoritmos de ordenamiento Bubble Sort optimizado, Insertion Sort, Selection Sort Optimizado y Cocktail Shaker Sort, junto con los algoritmos de búsqueda secuencial y búsqueda binaria iterativa. Además, se realizaron experimentos para analizar el comportamiento teórico y empírico de estos algoritmos en distintos tamaños de entrada y en diferentes casos de ejecución. Los resultados obtenidos permitieron comparar su rendimiento mediante benchmakrs y gráficos, facilitando la comprensión de sus diferencias en eficiencia y complejidad.

ÍNDICE GENERAL

Índice de figuras IV

1. Introducción 1

2. Objetivos 2
- 2.1. Objetivo Principal . . . 2
- 2.2. Objetivos Secundarios . . . 2

3. Marco teórico 3

4. Desarrollo 4
- 4.1. Diseño general del sistema . . . 4
  - 4.1.1. Estructura de datos de los deportistas . . . 4
  - 4.1.2. Generación y carga de datos . . . 4
  - 4.1.3. Funcionalidades del sistema . . . 5
- 4.2. Implementación de algoritmos de ordenamiento . . . 6
  - 4.2.1. Insertion Sort . . . 6
  - 4.2.2. Selection Sort optimizado . . . 6
  - 4.2.3. Cocktail Shaker Sort . . . 6
- 4.3. Implementación de algoritmos de búsqueda . . . 7
  - 4.3.1. Búsqueda secuencial . . . 7
  - 4.3.2. Búsqueda binaria iterativa . . . 7
- 4.4. Benchmarks, generación y visualización de resultados . . . 8
  - 4.4.1. Diseño de benchmarks . . . 8
  - 4.4.2. Casos de prueba . . . 8
  - 4.4.3. Generación de gráficos . . . 9
- 4.5. Análisis de resultados . . . 10
  - 4.5.1. Benchmark de búsqueda: Peor caso . . . 10
  - 4.5.2. Benchmark de ordenamiento: Mejor caso . . . 11
  - 4.5.3. Benchmark de ordenamiento: Caso promedio . . . 12
  - 4.5.4. Benchmark de ordenamiento: Peor caso . . . 13
  - 4.5.5. Análisis general . . . 14

5. Conclusión 15

ÍNDICE DE FIGURAS

4.1. Benchmark de búsqueda en peor caso 10
4.2. Benchmark de ordenamiento en mejor caso 11
4.3. Benchmark de ordenamiento en caso promedio 12
4.4. Benchmark de ordenamiento en peor caso 13

INTRODUCCIÓN

En el área de Diseño de Algoritmos, los métodos de ordenamiento y búsqueda son herramientas importantes para trabajar con conjuntos de datos de manera eficiente. Sin embargo, su importancia no solo está en conocer su funcionamiento teórico, sino también en observar cómo se comportan al aplicarlos sobre datos reales y en distintos escenarios de ejecución como el peor, el mejor y el caso promedio.

En esta tarea se desarrolló un sistema en lenguaje C para gestionar información de deportistas. El programa permite generar registros de manera automática, almacenarlos en archivos CSV, cargarlos para su procesamiento, ordenarlos según distintos campos y realizar búsquedas por ID, además de incorporar la opción de mostrar rankings de deportistas utilizando el puntaje como criterio principal.

Para resolver el problema, se implementaron los algoritmos de ordenamiento Bubble Sort optimizado, Insertion Sort, Selection Sort con optimización de intercambios innecesarios y Cocktail Shaker Sort, junto con búsqueda secuencial y búsqueda binaria iterativa. A partir de estas implementaciones, también se realizaron pruebas experimentales para comparar sus tiempos de ejecución en distintos tamaños de entrada.

El propósito de este informe es presentar el desarrollo del sistema, describir los algoritmos utilizados y analizar sus resultados teóricos y experimentales. De esta forma, se busca relacionar la implementación práctica con el comportamiento de cada algoritmo, identificando sus diferencias según el caso elegido.

OBJETIVOS

2.1. Objetivo Principal

Desarrollar un sistema en C para la generación, almacenamiento, ordenamiento y búsqueda de información de deportistas, utilizando algoritmos iterativos clásicos, con el fin de analizar su comportamiento tanto teórico como experimental sobre distintos tamaños de entrada.

2.2. Objetivos Secundarios

- Implementar correctamente los algoritmos de ordenamiento y sus versiones optimizadas según corresponda.
- Implementar correctamente los algoritmos de búsqueda.
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados.
- Evaluar y comprender la complejidad teórica de los algoritmos de ordenamiento y búsqueda.
- Documentar de manera adecuada el código desarrollado y los resultados obtenidos en el análisis experimental.

MARCO TEÓRICO

Los algoritmos de ordenamiento y búsqueda son fundamentales en el procesamiento de datos, ya que permiten organizar información y acceder a ella de manera eficiente. Su estudio no solo considera la forma en que funcionan, sino también el costo computacional asociado a su ejecución, el cual suele analizarse en función del tamaño de la entrada.

En este trabajo se utilizan algoritmos de ordenamiento iterativos clásicos: Bubble Sort, Insertion Sort, Selection Sort y Cocktail Shaker Sort. Estos métodos permiten reordenar los registros de deportistas según distintos criterios, como ID, nombre, equipo, puntaje o cantidad de competencias. Aunque todos cumplen la misma función general, presentan diferencias en su rendimiento dependiendo de si la entrada se encuentra ordenada, desordenada o invertida.

Por otra parte, para la localización de registros se emplean búsqueda secuencial y búsqueda binaria iterativa. La búsqueda secuencial revisa los elementos uno a uno hasta encontrar el valor buscado o llegar al final del arreglo, mientras que la búsqueda binaria divide repetidamente el espacio de búsqueda, por lo que requiere que los datos estén previamente ordenados.

El análisis de complejidad permite estimar el comportamiento de estos algoritmos en mejor, peor y caso promedio. Esta comparación teórica sirve como base para interpretar los resultados experimentales obtenidos durante la ejecución de los benchmarks realizados en el sistema.

DESARROLLO

4.1. Diseño general del sistema

El sistema fue diseñado para gestionar información de deportistas de forma simple y modular, permitiendo generar datos, cargarlos desde archivos CSV y aplicar sobre ellos distintas operaciones de ordenamiento, búsqueda y análisis experimental. Para facilitar su desarrollo y comprensión, la implementación se organizó en componentes que separan la representación de los datos, la generación y carga de registros, y las funcionalidades principales ofrecidas por el programa.

4.1.1. Estructura de datos de los deportistas

La estructura de datos utilizada para representar a cada deportista se definió como una estructura con campos específicos para almacenar distinta información; estos campos corresponden a:

- `ID`: Identificador único del deportista.
- `Nombre`: Nombre completo del deportista.
- `Equipo`: Equipo al que pertenece.
- `Puntaje`: Puntuación total acumulada.
- `Competencias`: Número de competencias en las que ha participado.

Cada parámetro permite almacenar información, la cual puede ser utilizada para ordenar o buscar a los deportistas según distintos criterios. Esta estructura se implementó utilizando un `struct` en C, lo que facilita su manejo y acceso a los campos correspondientes durante las operaciones de ordenamiento y búsqueda.

4.1.2. Generación y carga de datos

La generación de datos de los parámetros asociados a deportistas se realizó mediante distintas funciones, las cuales permiten crear u obtener estos registros de manera .ªleatoria". La generación de un nombre se realizó permitiendo seleccionar aleatoriamente letras de un conjunto predefinido, escogiendo un tamaño de 3 a 8 caracteres. Para el equipo, se seleccionó

aleatoriamente entre un conjunto específico de equipos de Esports. El ID se generó en forma incremental, asegurando la unicidad de cada registro. Por otro lado, el puntaje y la cantidad de competencias se generaron utilizando funciones de generación de números aleatorios dentro de rangos predefinidos. Esta metodología permitió crear un conjunto de datos variado y representativo para probar las funcionalidades del sistema. Posterior a la creación de los registros, estos se desordenaron utilizando el algoritmo de Fisher-Yates para garantizar que los datos no se encuentren en un orden específico, lo que es fundamental para evaluar correctamente el desempeño de los algoritmos de ordenamiento y búsqueda implementados. Finalmente, los registros generados se almacenaron en un archivo CSV, lo que facilita su posterior carga y manipulación dentro del programa.

4.1.3. Funcionalidades del sistema

Una vez cargados los datos de los deportistas, el programa permite realizar distintas operaciones sobre ellos por medio de parámetros de línea de comandos. Entre las funcionalidades principales se encuentra el ordenamiento de los datos de los deportistas de manera ascendente o descendente utilizando el ID, puntaje, cantidad de competencias, nombre y equipo. Para ello, el usuario puede seleccionar alguno de los algoritmos de ordenamiento implementados en el programa.

Otra funcionalidad importante es la búsqueda de deportistas por ID, la cual puede realizarse mediante búsqueda secuencial o búsqueda binaria iterativa. Esto permite comparar no solo la correctitud de ambos métodos, sino también su desempeño bajo distintas condiciones de entrada.

Además, el sistema permite mostrar un ranking con los mejores N deportistas de acuerdo a su puntaje, lo que logra una aplicación práctica del ordenamiento de datos dentro del contexto del problema planteado.

Junto con las operaciones anteriores, el programa incorpora la ejecución de benchmarks para medir el tiempo de ejecución de los algoritmos de ordenamiento y búsqueda. Estos resultados se almacenan en archivos CSV y posteriormente pueden utilizarse para generar gráficos comparativos mediante gnuplot, facilitando el análisis experimental de los algoritmos implementados.

En conjunto, estas funcionalidades permiten que el sistema no solo cumpla con los requerimientos prácticos de manipulación de datos, sino que también sirva como base para el estudio comparativo del comportamiento de los algoritmos.

4.2. Implementación de algoritmos de ordenamiento

Para el procesamiento de los datos de deportistas, el sistema incorpora distintos algoritmos de ordenamiento iterativos, cada uno con características particulares en cuanto a funcionamiento y rendimiento. Su implementación permite ordenar los datos según diversos criterios y, al mismo tiempo, comparar su comportamiento en distintos casos de prueba desde un punto de vista teórico y experimental.

4.2.1. Insertion Sort

Insertion Sort es un algoritmo de ordenamiento iterativo que construye gradualmente una porción ordenada del arreglo. Su funcionamiento consiste en tomar cada elemento y ubicarlo en la posición que le corresponde dentro de los elementos previamente ordenados, desplazando hacia la derecha aquellos que sean mayores o menores según el criterio de orden elegido.

En este proyecto, su implementación permite ordenar los registros de deportistas de acuerdo con distintos campos, como ID, puntaje, competencias, nombre y equipo.

Una de las ventajas de Insertion Sort es su buen desempeño cuando los datos ya se encuentran ordenados o casi ordenados, ya que en esos casos realiza pocos desplazamientos. En cambio, cuando la entrada se encuentra en el peor caso, su cantidad de comparaciones y movimientos aumenta considerablemente.

Teóricamente, Insertion Sort presenta complejidad $O(n)$ en el mejor caso y $O(n^2)$ tanto en el caso promedio como en el peor caso.

4.2.2. Selection Sort optimizado

4.2.3. Cocktail Shaker Sort

Este algoritmo corresponde a una variación de Bubble Sort (4.2), la cual altera su rendimiento en ciertos casos. Cocktail Shaker Sort, o Bidirectional Bubble Sort, actúa recorriendo el arreglo en ambas direcciones durante cada iteración, es decir, desde izquierda a derecha y luego de derecha a izquierda. Esto permite que los elementos más grandes se desplacen hacia el final del arreglo y los más pequeños hacia el inicio de manera más eficiente, especialmente cuando la entrada se encuentra en un orden parcialmente invertido. Sin embargo, en el mejor caso, cuando los datos ya están ordenados, su rendimiento se ve afectado por la necesidad de realizar dos pasadas completas para verificar que el arreglo está ordenado, lo que resulta en un tiempo de ejecución mayor en comparación con otros algoritmos como Insertion Sort

o Bubble Sort optimizado. En el caso promedio y peor caso, Cocktail Shaker Sort mantiene una complejidad $O(n^2)$, aunque su rendimiento puede ser mejor que Bubble Sort en ciertos escenarios debido a su capacidad para mover elementos en ambas direcciones. Al igual que los demás algoritmos de ordenamiento implementados en este proyecto, Cocktail Shaker Sort es capaz de ordenar los registros de deportistas según distintos criterios y sentidos, lo que permite evaluar su desempeño en función de la disposición inicial de los datos y compararlo con otras estrategias de ordenamiento.

4.3. Implementación de algoritmos de búsqueda

Además de ordenar los datos, el sistema permite localizar deportistas específicos a partir de su ID mediante algoritmos de búsqueda iterativos. Para ello se implementaron dos enfoques distintos, lo que permite comparar una estrategia de búsqueda directa sobre datos no ordenados con otra más eficiente que requiere un arreglo previamente ordenado.

4.3.1. Búsqueda secuencial

La búsqueda secuencial consiste en recorrer los elementos del arreglo uno a uno hasta encontrar el valor buscado o llegar al final de la estructura. En este proyecto, su implementación se utiliza para localizar deportistas a partir de su ID, comparando el identificador buscado con el ID de cada registro almacenado en memoria.

Una de las principales ventajas de este método es su simplicidad, ya que no requiere que los datos se encuentren previamente ordenados. Esto la convierte en una alternativa directa para trabajar sobre el arreglo cargado desde el archivo CSV. Sin embargo, su rendimiento depende de la posición del elemento buscado, y en el peor caso debe revisar todos los registros antes de concluir que el deportista existe o no existe en el arreglo.

Desde el punto de vista teórico, su complejidad temporal en el peor caso es $O(n)$, ya que el número de comparaciones crece linealmente con la cantidad de elementos.

4.3.2. Búsqueda binaria iterativa

La búsqueda binaria iterativa es un algoritmo eficiente para localizar elementos en un arreglo previamente ordenado. Su funcionamiento se basa en dividir el espacio de búsqueda en cada iteración, comparando el valor buscado con el elemento central del arreglo. Si el valor coincide, se ha encontrado el elemento; si el valor es menor, se continúa la búsqueda en la mitad inferior del arreglo; y si es mayor, se busca en la mitad superior. Este proceso se repite hasta encontrar el elemento o determinar que no está presente en el arreglo. El tiempo

de ejecución de la búsqueda binaria iterativa es $O(\log n)$ en el peor caso, lo que la convierte en una opción significativamente más rápida que la búsqueda secuencial para arreglos grandes, siempre y cuando los datos estén ordenados previamente. La implementación de este algoritmo en el proyecto permite localizar deportistas únicamente por su ID, lo que requiere que el arreglo se encuentre ordenado según este campo antes de realizar la búsqueda. Sin embargo, para los tiempos de ejecución se midió su comportamiento sin contar el tiempo de ordenamiento previo, lo que permite evaluar su eficiencia exclusivamente en la fase de búsqueda.

4.4. Benchmarks, generación y visualización de resultados

4.4.1. Diseño de benchmarks

Con el fin de evaluar empíricamente la eficiencia de los algoritmos implementados, el sistema incorpora benchmarks para ordenamiento y búsqueda. Estas pruebas permiten medir los tiempos de ejecución en distintos tamaños de entrada, utilizando múltiples repeticiones por cada tamaño con el fin de reducir la variabilidad de los resultados.

En el benchmark de ordenamiento se consideran los cuatro algoritmos implementados: Bubble Sort optimizado, Insertion Sort, Selection Sort optimizado y Cocktail Shaker Sort. En el benchmark de búsqueda se evalúan búsqueda secuencial y búsqueda binaria. Los tiempos obtenidos se almacenan en archivos CSV, lo que facilita su análisis y representación gráfica.

El diseño de estas pruebas busca relacionar el comportamiento observado en la práctica con la complejidad teórica de cada algoritmo, permitiendo comparar sus diferencias de rendimiento bajo condiciones controladas.

4.4.2. Casos de prueba

Para los algoritmos de ordenamiento se consideraron tres escenarios de entrada: mejor caso, caso promedio y peor caso. El mejor caso se representó mediante arreglos previamente ordenados en forma ascendente según ID, el caso promedio mediante arreglos mezclados aleatoriamente y el peor caso mediante arreglos ordenados en forma descendente. Esta separación permite observar cómo cambia el rendimiento de cada algoritmo según la disposición inicial de los datos.

En el caso de los algoritmos de búsqueda, se evaluó el peor caso. Para búsqueda secuencial, este escenario se simuló utilizando un ID inexistente, obligando al algoritmo a recorrer

completamente el arreglo antes de concluir que el elemento no está presente. Para búsqueda binaria, se utilizó igualmente un ID inexistente, pero sobre un arreglo previamente ordenado, asegurando así una medición consistente de su comportamiento en este caso.

Gracias a estos casos de prueba, fue posible comparar de manera más precisa el desempeño de cada algoritmo y relacionarlo con lo esperado desde el análisis teórico.

4.4.3. Generación de gráficos

Los resultados obtenidos en los benchmarks se almacenaron en archivos CSV, los cuales posteriormente fueron utilizados para generar gráficos comparativos mediante `gnuplot`. Esta etapa permitió representar visualmente la evolución de los tiempos de ejecución en función del tamaño de entrada, facilitando la interpretación de los resultados experimentales.

Para la búsqueda se generó un gráfico correspondiente al peor caso, mientras que para los algoritmos de ordenamiento se elaboraron gráficos independientes para mejor caso, caso promedio y peor caso. De esta forma, fue posible comparar con mayor claridad las diferencias de comportamiento entre los algoritmos en cada escenario analizado.

En particular, para el gráfico del mejor caso de los algoritmos de ordenamiento se utilizó una escala logarítmica en el eje vertical. Esta decisión se tomó debido a que, en dicho escenario, algunos algoritmos presentan tiempos de ejecución muy cercanos entre sí y considerablemente menores que otros, lo que dificulta apreciar visualmente sus diferencias en una escala lineal. El uso de escala logarítmica permitió distinguir de manera más clara estas variaciones y realizar una comparación más significativa entre los algoritmos.

4.5. Análisis de resultados

4.5.1. Benchmark de búsqueda: Peor caso

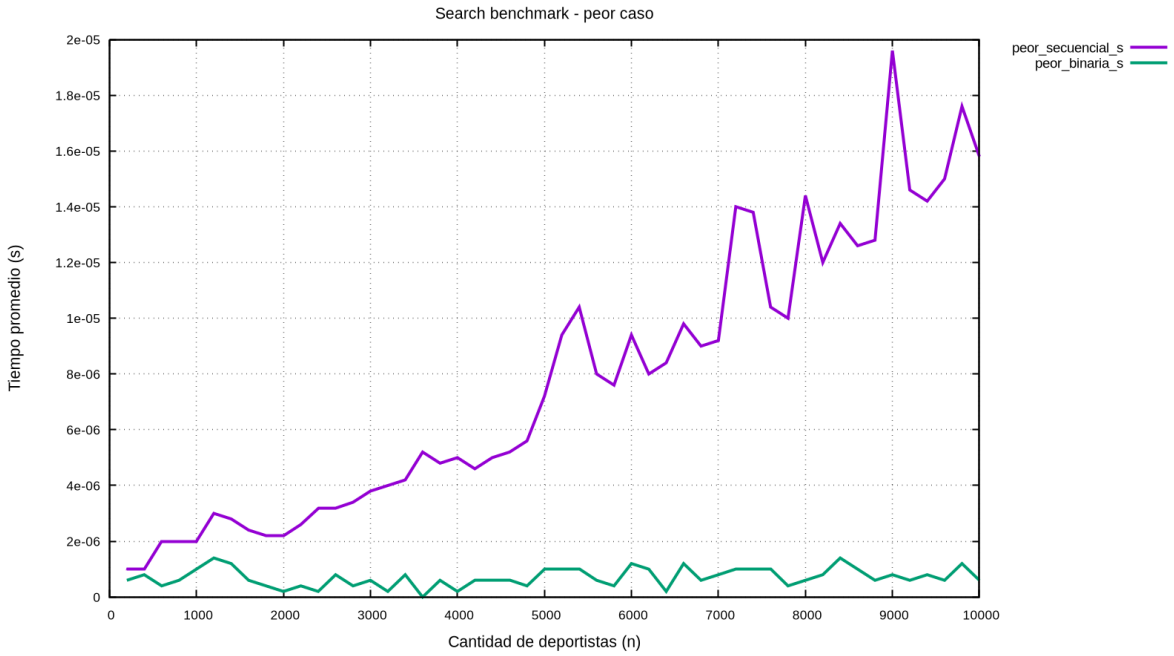

Figura 4.1: Benchmark de búsqueda en peor caso

En la Figura 4.1 se aprecia que la búsqueda secuencial presenta una mayor demora que la búsqueda binaria en el peor caso. Esto coincide con la complejidad temporal esperada, ya que la búsqueda secuencial tiene costo $O(n)$ al revisar todos los elementos cuando el ID buscado no existe, mientras que la búsqueda binaria mantiene un costo $O(\log n)$ al dividir el espacio de búsqueda en cada iteración. Por ello, a medida que aumenta la cantidad de datos, la diferencia entre ambos algoritmos se vuelve más evidente y confirma la ventaja de la búsqueda binaria sobre arreglos ordenados.

4.5.2. Benchmark de ordenamiento: Mejor caso

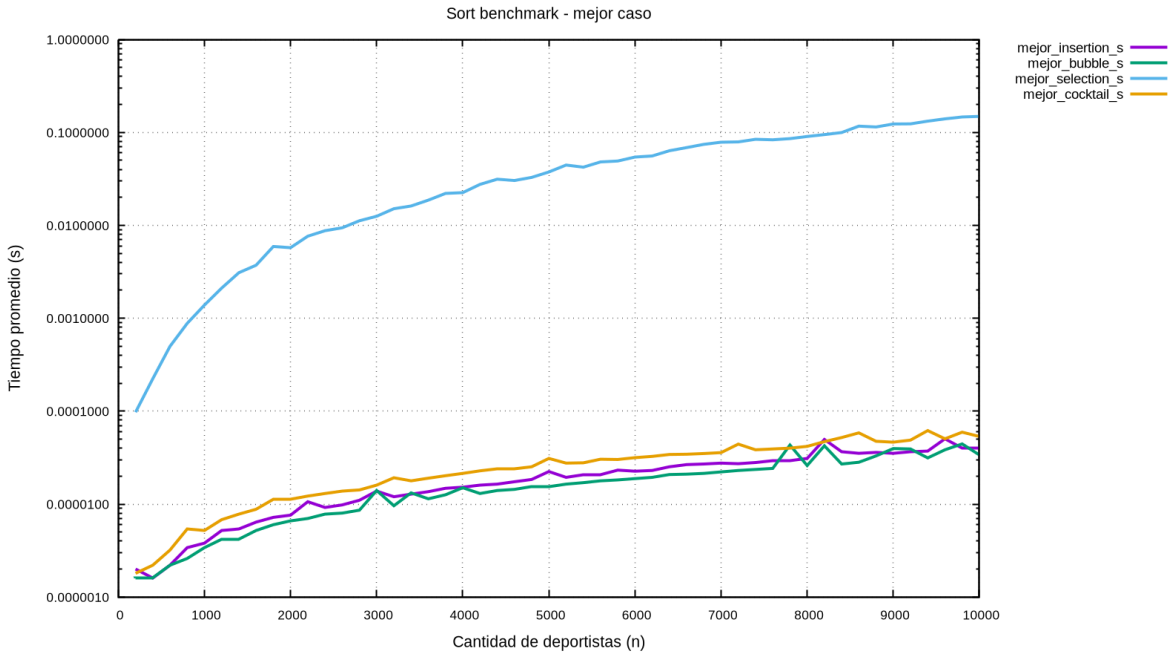

Figura 4.2: Benchmark de ordenamiento en mejor caso

En la Figura 4.2, Bubble Sort optimizado e Insertion Sort presentan los menores tiempos de ejecución, seguidos por Cocktail Shaker Sort, mientras que Selection Sort queda claramente por encima del resto. Este comportamiento es coherente con la teoría: Insertion Sort tiene complejidad $O(n)$ en el mejor caso, y Bubble Sort optimizado también se acerca a $O(n)$ al detectar rápidamente que el arreglo ya está ordenado. Cocktail Shaker Sort mejora notablemente en este escenario, aunque conserva una leve sobrecarga adicional. En cambio, Selection Sort mantiene un costo $O(n^2)$, ya que continúa realizando la mayoría de sus comparaciones aunque los datos ya estén ordenados.

4.5.3. Benchmark de ordenamiento: Caso promedio

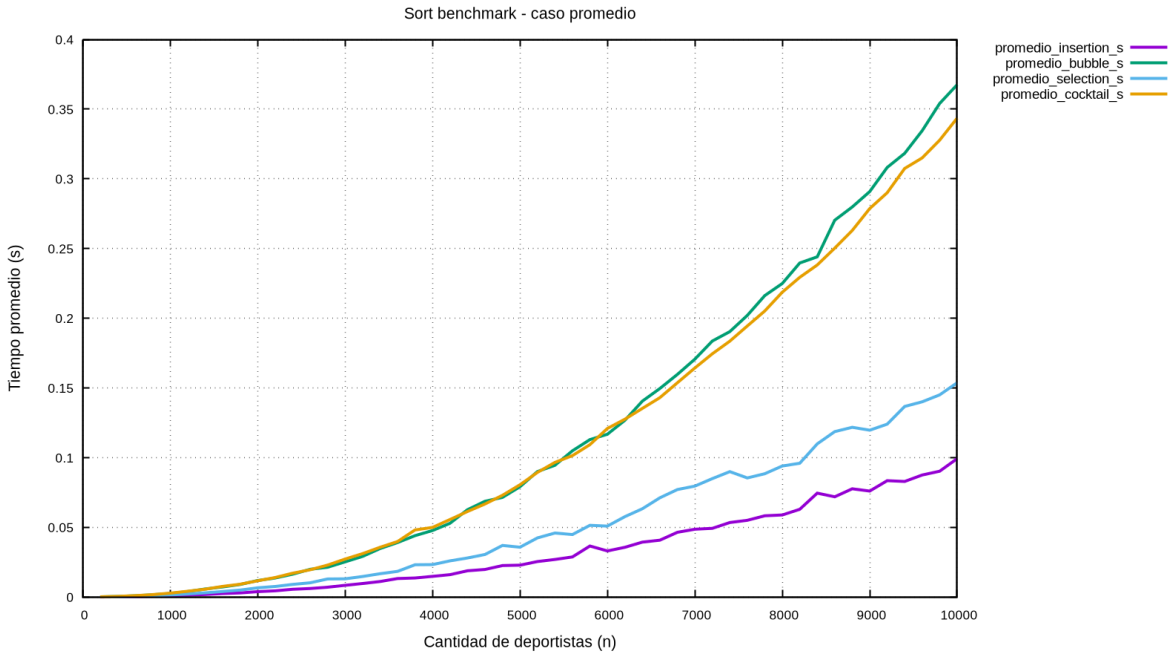

Figura 4.3: Benchmark de ordenamiento en caso promedio

La Figura 4.3 muestra que Insertion Sort obtiene el mejor rendimiento en el caso promedio, seguido por Selection Sort, mientras que Bubble Sort optimizado y Cocktail Shaker Sort presentan tiempos mayores. En este escenario, los cuatro algoritmos tienen complejidad temporal $O(n^2)$, pero se diferencian en la cantidad de comparaciones e intercambios realizados. Insertion Sort logra un mejor comportamiento práctico al insertar cada elemento en su posición de forma relativamente eficiente, mientras que Selection Sort se mantiene competitivo por efectuar pocos intercambios. Por su parte, Bubble Sort y Cocktail Shaker Sort aumentan más rápidamente su tiempo de ejecución debido a la mayor cantidad de movimientos necesarios sobre datos desordenados.

4.5.4. Benchmark de ordenamiento: Peor caso

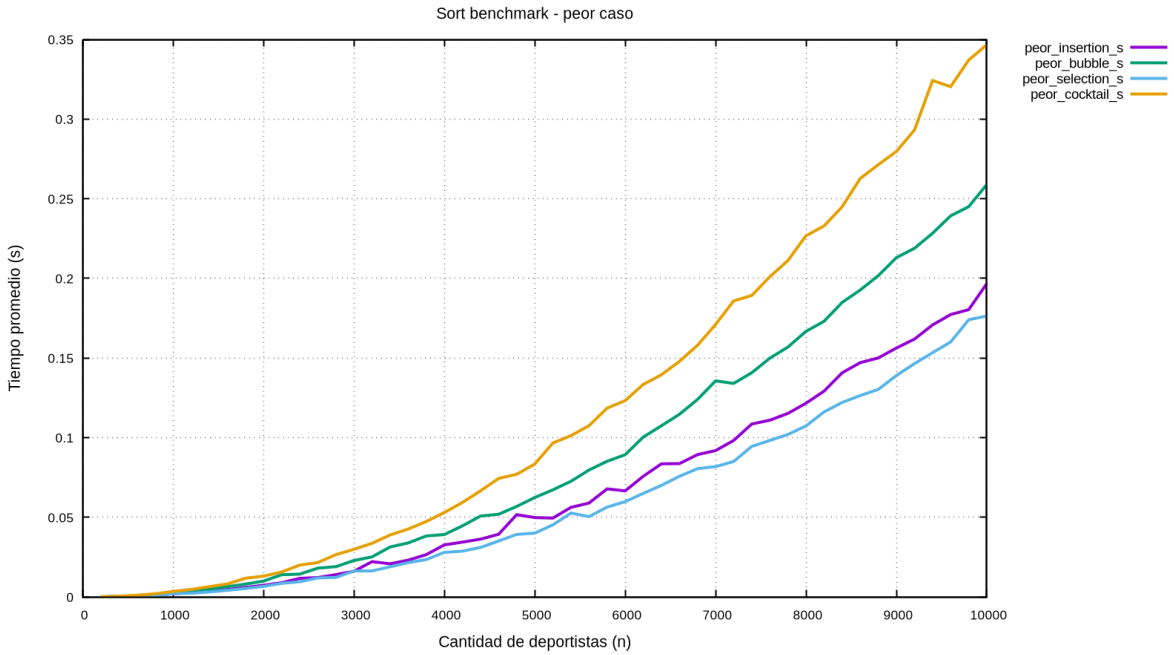

Figura 4.4: Benchmark de ordenamiento en peor caso

En la Figura 4.4 se observa que Selection Sort presenta los menores tiempos dentro del peor caso, seguido por Insertion Sort, mientras que Bubble Sort optimizado y Cocktail Shaker Sort son los más lentos, siendo este último el de mayor demora. Teóricamente, los cuatro algoritmos conservan complejidad $O(n^2)$ en este escenario, pero sus constantes de ejecución son distintas. Selection Sort mantiene un comportamiento más estable porque realiza pocos intercambios, incluso cuando la entrada está invertida. Insertion Sort empeora por el aumento de desplazamientos necesarios, mientras que Bubble Sort y Cocktail Shaker Sort exhiben un crecimiento mayor debido a la gran cantidad de comparaciones e intercambios requeridos para reordenar completamente el arreglo.

4.5.5. Análisis general

En términos generales, los resultados experimentales obtenidos son coherentes con el comportamiento teórico esperado para los algoritmos implementados. En el caso de búsqueda, la búsqueda binaria mostró un mejor rendimiento que la búsqueda secuencial en el peor caso, lo que coincide con sus complejidades $O(\log n)$ y $O(n)$, respectivamente. Esto confirma que, cuando los datos se encuentran ordenados, la búsqueda binaria resulta considerablemente más eficiente para tamaños de entrada crecientes.

En los algoritmos de ordenamiento también se observaron tendencias acordes a la teoría. En el mejor caso, Bubble Sort optimizado e Insertion Sort presentaron los menores tiempos, mientras que Selection Sort se mantuvo claramente por encima debido a que continúa realizando un gran número de comparaciones incluso cuando la entrada ya está ordenada. En el caso promedio, Insertion Sort obtuvo el mejor desempeño general, seguido por Selection Sort, mientras que Bubble Sort y Cocktail Shaker Sort presentaron mayores tiempos de ejecución. Finalmente, en el peor caso, Selection Sort mostró un comportamiento más estable, mientras que Bubble Sort y Cocktail Shaker Sort concentraron las mayores demoras.

En conjunto, estos resultados permiten concluir que no todos los algoritmos con la misma complejidad asintótica presentan el mismo rendimiento en la práctica. Aunque varios de ellos comparten complejidad $O(n^2)$, las diferencias en la cantidad de comparaciones, desplazamientos e intercambios afectan directamente su desempeño real. De esta manera, el análisis experimental complementa la teoría y permite entender con mayor claridad las ventajas y limitaciones de cada algoritmo en distintos escenarios.

CONCLUSIÓN